JOURNAL OF AEROSPACE INFORMATION SYSTEMS
Vol. 15, No. 5, May 2018

From Natural Language Standard Documents to State Machines: Advantages and Drawbacks

Juliana Galvani Greghi*
University of Lavras, CEP 37200-000 Lavras, Minas Gerais, Brazil
Eliane Martins† and Ariadne M. B. R. Carvalho†
University of Campinas, CEP 13083-852 Campinas, São Paulo, Brazil
Ana Maria Ambrosio‡
National Institute of Space Research, CEP 12227-010, São José dos Campos, São Paulo, Brazil
and
Emília Villani§
Aeronautics Institute of Technology, CEP 12228-900, São José dos Campos, São Paulo, Brazil

DOI: 10.2514/1.I010525

Problems in requirements documents are among the root cause of a number of accidents in space missions. A common approach toward the minimization of these problems is to transform the requirements into models that represent the system's behavior. However, this solution requires dealing with issues such as choosing the best modeling formalism, defining to what extent the transformation process should be automated, and assuring the quality of the requirements documents to be used as input. In space missions, requirements are frequently tailored from standard documents, such as the Packet Utilization Standard, which are composed of mandatory and optional requirements. This paper presents a semi-automatic method to transform standard requirements documents into extended finite state machines. To evaluate it, we apply the method to a set of requirements from the Packet Utilization Standard. We evaluate the method using some Packet Utilization Standard services. In light of the results, the paper discusses advantages and potential problems of each approach.

I. Introduction

IN THE conservative area of big and medium satellites, requirements documents are still written in natural language. A common approach to improve the quality of the requirements document is to transform it into models. However, most models are manually generated by analysts, an activity that requires time and experience. When creating models, the analyst adds his/her own interpretation of the information on the documents. The analyst may use his/her daily experience and include common sense information that is not described in the text. Consequently, different people may create different models from the same requirements document. Manual modeling is an error-prone task [1] due to human interference.

Lutz and Ampo [2] show that some catastrophic failures in space missions were consequences of problematic requirements. In an attempt to minimize these failures, agencies such as the European Cooperation for Space Standardization (ECSS) proposed standards that define requirements, specifications, and processes for space products and their systems. To accommodate the needs of different missions, these standards are usually composed of mandatory and optional requirements. The set of requirements of a mission is composed of all the mandatory requirements plus a subset of the optional requirements. Different sets of requirements tailored from the same standard correspond to different models. The more complex the set of requirements, the more time consuming its translation into a model is.

In an attempt to minimize problems during the translation process, several automatic and semi-automatic approaches have been proposed to generate functional and behavioral models, such as [3–5]. Because the quality of the requirements documents in natural language has been appointed as a key factor to the information extraction process, some approaches recommend rewriting the text before its translation, in order to remove ambiguities, inconsistencies, and other problems related to the poor quality of the document. Other approaches propose restructuring the document in predefined formats, like use cases. However, when the document under analysis is a worldwide standard document, these approaches may not be feasible since the original texts might not be rewritten. This is the case of requirements documents derived from the ECSS standards, applied to space missions in the entire world. In these cases, the approach must deal with the requirement document as is. It must also be able to accommodate the variability issues that arise from the tailoring of the standard for a given mission, i.e., the possibility of including or not including in the models the optional requirements of that mission.

To approach these challenges, this paper presents a semi-automatic method to generate Extended Finite State Machines (EFSMs) from standard documents. A Finite State Machine (FSM) is a transition graph commonly used to describe discrete event systems, such as computational systems. EFSMs extend FSMs, allowing data variables and functions associated to transitions. Formal definitions of FSMs and EFSMs are presented in Sec. III.A.

The paper also introduces TXT2SMM, a tool developed to support the proposed method. It illustrates the functionality and the applicability of the method and the use of the TXT2SMM tool through its application to one of the services of the Packet Utilization Standard (PUS) [6] provided by the ECSS. This service standardizes the verification of telecommands onboard satellites. EFSMs obtained from the semi-automatic method are compared with FSM models, manually created by specialists for the same PUS service and available in the literature [7–9]. FSMs are used because, to the best of our knowledge, no EFSM models have been published so far for any PUS service.

Received 30 December 2016; revision received 25 December 2017; accepted for publication 26 January 2018; published online 28 March 2018.

*Department of Computer Science, Campus Universitário, P.O. Box 3037.
†Institute of Computing, Av. Albert Einstein, 1251 Cid. Univ. Zeferino Vaz CEP 13083-852.
‡National Institute for Space Research, Av. dos Astronautas, 1758, Jd. Granja CEP 12227-010.
§Competence Center for Manufacturing, Pr. Mal. Eduardo Gomes, 50–V. das Acácias, CEP 12228-900.

The comparison takes into account the similarities and differences in the structure of state machines and the coverage and adherence to the requirements of the PUS service. Although FSMs and EFSMs are different modeling formalisms, they share the same structure of states, transitions, and alphabet.

The evaluation presented in our previous works ([10,11]) has shown that the models have similarities and might be useful for analysts. In this work, we extend the previous evaluation and focus on the differences between the manually and the semi-automatically generated models as well as on the differences among the manually created models described in [7–9]. We also discuss the advantages of the semi-automatic process to generate models that can be used for validating an implementation of the PUS services [12].

The remainder of the paper is organized as follows. Section II discusses related work. Section III describes the proposed method and the TXT2SMM tool, developed as a proof of concept. A controlled experiment and the comparison between manually and semi-automatically generated models are presented in Sec. IV. Section V analyzes the results from the experiment. Finally, Sec. VI draws some conclusions and discusses future work.

II. Background

Standard documents may have inconsistencies and ambiguity problems similarly to ordinary requirements documents. According to Berry and Kamsties [13], natural language is inherently ambiguous, and even with attentive writers, these problems may occur. The techniques and methodologies considered in the scope of this work were classified in two categories: i) those that make changes in the text before model generation [1,4] and ii) those that keep the text as is [3,5]. In [1], an environment for natural language requirements analysis called CIRCE is presented. CIRCE is based on Natural Language Processing (NLP) tools and techniques, and the user may visualize the information in order to complete or correct the original requirements. Deeptimanhanti and Babar [4] and Deeptimanhanti and Sanyal [14] developed a tool named UMGAR, which uses many NLP tools and techniques and normalizes sentences to eliminate ambiguities before generating Unified Modeling Language (UML) analysis models. Our interest is in techniques that do not alter the content of the text. They include approaches that generate models from structured documents, such as use cases [3], from a natural language description of an automaton [5], and from natural language requirements documents [15]. In addition to research involving model generation, some of them apply Software Product Line (SPL) concepts to the modeling problem. Ali et al. [16] present a product line model and a configuration method to support model-based tests. The method uses predefined models, divided into four categories, to create different model configurations to be used to test different products. Yue et al. [3,17] developed an approach to generate UML analysis models from use cases structured according to Restricted Use Case Modeling (RUCM) [18]. The approach was implemented as part of the aToucan tool [19], and the generated models were validated through a case study and a comparison with three commercial tools. In the case study, five different software descriptions were analyzed: three from textbooks and two created by Master's degree students. The use cases were rewritten, applying RUCM, and the validation aimed to find out if the transformation rules were complete and if the generated activity diagrams were syntactically and semantically correct. The authors also defined a validation procedure, applied to the entire process. They achieved a high degree of correctness and completeness, and traceability links were correctly established. However, aToucan was not able to correctly deal with the data flow information attached to each activity diagram. Regarding the comparison with commercial tools, aToucan presents additional features, for instance, the possibility of concurrency specification and support to include and extend relationships. The similarities between aToucan and the work presented here relies on the fact that both take a requirements document in natural language and extract information from the document to generate a model. However, the structure of the document used in aToucan is quite different from the document used here.

The approach presented by Kof [5] uses a natural language description of the system's behavior as input and generates finite automata. The generated automata were validated through a case study in which students were invited to manually generate models from a specification, which were in turn compared with the automatically generated ones.

Kof and Penzenstadler [15] present a method to integrate a NLP approach and a computer-aided software engineering tool. They introduce a methodology for translating text to a model using a previous work [5]. The process requires the user to select the section of the document to be processed. The methodology was integrated with a tool that enables user interaction. The tool learns on the fly about words and grammar constructions that can be used for model generation. All approaches previously cited illustrate case studies in which humans manually model a set of requirements to evaluate and validate the generated models. In our work, we compare models manually created by different teams at different times; then, we compare those models with the semi-automatically generated ones. Our input texts are standards the space area, written in natural language. As is usual with standards, they have been submitted to meticulous revision processes by multiple specialists. Nonetheless, they eventually may contain problems, such as ambiguities or incomplete information.

III. Method and TXT2SMM Tool

In this section, we will briefly present the method developed to analyze and extract information from standard documents and the TXT2SMM tool, developed as a proof of concept. More details about the method and the tool can be found in [10].

A. Modeling Formalism

The proposed method aims to generate an EFSM from textual requirements. The EFSM considered in this work is an extension of a Mealy machine, which is a FSM of which the output values are determined by both its current state and the current inputs. All FSMs described in this paper are Mealy machines. A Mealy machine is formally defined as a sextuple $M = (Q, I, O, T, S, q_0)$ [20], where we have the following:

1) Q is a finite set of states.
2) I is a finite set of input symbols, called the "input alphabet."
3) O is a finite set of output symbols, called the "output alphabet."
4) T is a transition function, a total function from $Q \times I$ to Q.
5) S is an output function, a total function from $Q \times I$ to O.
6) Finally, q_0 is the initial state.

An EFSM is an FSM with additional variables and enabling functions. It is formally defined as a 7-tuple $M = (Q, I, O, D, F, q_0)$ [21], where we have the following:

1) Q, I, O, q_0 have the same definition as for FSMs.
2) D is a set of variables.
3) F is a set of enabling functions defined over D.
4) T is a transition function, a total function from $Q \times I \times F$ to $Q \times O$.

B. Method

The method proposed in this paper consists of analyzing the standard documents and identifying patterns in the sentence and in the document structure to help during the information extraction process. NLP techniques are used to support patterns identification in the sentence structure. The method requires two human participants: the expert analyst, responsible for document analysis and identification of mandatory and optional requirements, and the expert user, who selects the services and functionalities that will be represented in the model, i.e., functionalities needed for a specific space mission. Therefore, different space missions can be specified from the same standard, according to the combination of selected mandatory and optional requirements, as it occurs in SPLs [22]. For that reason, the method is feature oriented, as in SPL development. A feature is an aspect or characteristic of the system considered important by its users, which, in our case, are the mandatory and optional requirements specified in the standard. The document analysis is organized in four major steps, already published in [10] and briefly described here.

1. *Step 1: Analysis of Standard Document*

In the first step, the expert analyst searches the standard document for evidence that patterns occur in the text sentences. Here, the analysis consists of having the standard document processed by a word-frequency counter tool. Highly frequent words, but with no semantic meaning, are discarded. Stop words [23] are examples of this kind of word. After finding evidence of patterns in the sentences, the next task is to identify information that points to external references, i.e., to other documents, or to a different section or chapter of the same document. This information might be necessary for model generation and must be registered for future processing; it is obtained through careful reading of the standard document.

In addition to external references, the expert analyst must search for information that is present only in tables or figures. This information is also manually extracted and registered for future processing.

The analysis of the document continues with the processing of the document with a Part of Speach (PoS) Tagger, a NLP tool responsible for attributing a category to every word in the text [24]. Sequences of tags were already used by Kof [5]. Nevertheless, the sequences used here are different because they are defined according to the standard document and to the service description under analysis. In addition to the categories, the structural pattern of the document, already identified, can be used during information extraction.

After the document structural patterns and the sentence patterns (which will help in the information extraction process) have been defined, the expert analyst must identify specific information: states and transitions. The identification of this information relies on the location of the information, e.g., on the section of the document where the information is described, and on the sequence of tags. In addition to the location and sequences of tags, the expert analyst may use the document structure to obtain information about the text; e.g., the order of the states in the text can be used to find the transition event. The information about the document, sentence structure, and sequence of tags is used to generate rules for extracting information from the standard document. It is important to clarify that the manual analysis is necessary because the objective of the document is not to describe a service's automaton and therefore it does not follow a predefined structure to describe states and transitions. So, the expert analyst must read the document and identify parts of the text that are describing, for instance, states.

2. *Step 2: Identification of Mandatory and Optional Requirements*

Analyzed the document in search of information to be extracted, the expert analyst must identify mandatory and optional requirements described in the text. These requirements, in general, describe mandatory functionalities, i.e., functionalities that must be always represented in an implementation of the described service, and optional functionalities, which are implemented on demand.

Following the feature-oriented domain analysis method [22], these functionalities are represented in a feature diagram, a treelike structure used to represent mandatory and optional features (and their relationships) in a SPL [22]. For the time being, the expert analyst manually extracts the feature diagram from the PUS standard.

3. *Step 3: Selection of Service and Functionalities to be Modeled*

The service to be modeled, described in the PUS standard document, is manually selected by an expert user. The corresponding mandatory functionalities are selected by default, and the optional functionalities are presented to the user. The information about whether a functionality is mandatory or not is provided by the feature diagram elaborated in the previous step.

4. *Step 4: Information Extraction*

Information extraction is an automated task, which uses the set of rules created during the analysis of the document (steps 1, 2, and 3). The rules are applied to the standard document to extract information about the selected service. After the extraction process is completed, the relevant information is filtered and used for model generation. The automated tasks are executed with the TXT2SMM tool.

C. TXT2SMM Tool

The TXT2SMM tool was developed to support the application of the method. It also serves as a proof of concept of the method feasibility. The feature diagram, elaborated by the analyst in step 2, is used to create rules for model generation based on the features' relationship. The information extraction process is performed in the standard document, preprocessed by a PoS Tagger. After the information has been extracted and the service and functionalities have been selected, the resultant model is delivered in .sm file extension format, a specific representation for state machines, proposed in [25]. The second step of the method (presented in Sec. III.B), indicates that the requirements variability must be represented with a feature model. We briefly describe how to identify them.

In this paper, features were defined according to the expert user's viewpoint. The feature diagram presented in [11] included features that are related to the user's choice but were external to TXT2SMM. In this new version, we represent only the user's choices internal to the tool, as in [26]. In Fig. 1, the feature diagram is used to represent the commonalities and variabilities of the TXT2SMM tool.

The generation of a new model requires the configuration of four main mandatory features. They are mandatory because these features represent the minimal choices to configure the service and the features to be modeled.

The user must select the document. This feature is represented by Document Version in an alternative relationship with the subfeatures V1 and Vn, meaning the user must select only one version of the standard document. Service Description presents the or-features S1 and Sn, which means that at least one service must be selected. Each service has a set of subfeatures, representing mandatory and optional functionalities. Mandatory functionalities are represented by Functionality Group 1, and additional functionalities are represented by optional features in Functionality Group 2 and Functionality Group N. Each functionality may describe a set of subfunctionalities, such as reports, messages, or requests, represented by Functionality 1 to Functionality N, which, in turn, may be mandatory, optional, alternative, or or-feature, depending on the requirements of each functionality. As described in step 1 (Sec. III.B), sets of rules are created for information extraction, and each set is specific

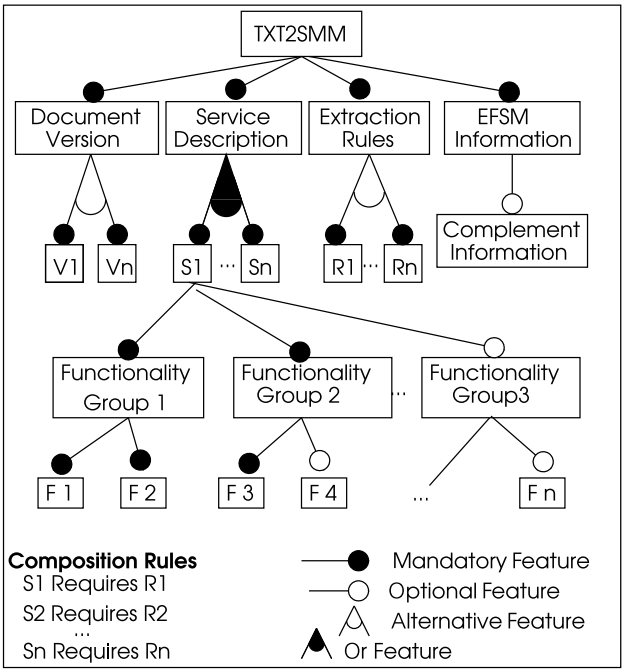


Fig. 1 Feature diagram for TXT2SMM.

for a service. The mandatory feature Extraction Rules represents the several sets of rules, defined for each service, and the alternative features R1 and Rn represent each set that can be selected taking into account that only one of them must be selected.

The mandatory feature EFSM Information represents the information extracted for state machine generation. The possibility for the user to edit it, complementing information with mission's requirements, is represented by the optional feature Complement Information.

IV. Controlled Experiment and Comparison

The proposed approach was applied to the PUS document [6], which describes a set of 16 services to support functionalities for spacecraft onboard data handling. Each service is described with a minimum capability set that must be always included in every implementation of the service and additional capability sets that may be optionally implemented. The set of services to be implemented depends on the space mission requirements. The document also defines other important requirements that must be taken into account in the implementation of each service and that appear in different sections of the PUS document. All these aspects affect variability. The text was preprocessed with the Stanford PoS Tagger [24], and the method was applied to the Telecommand Verification Service, described in the standard document. This service is responsible for verifying a telecommand received onboard. The mandatory features of this service are related to the generation of a telecommand acceptance report, whereas optional features are related to additional reports that indicate the success or failure of the telecommand execution phases (start, progress, and completion).

The comparisons were made between pairs of models, always a manual and a semi-automatic generated model. It is not the intention here to determine whether the machines are equivalent in behavior or are a refinement of each other, as these would require appropriate tools and a more complex analysis. The goal here is simply to detect similarities and differences in the structure of state machines. Although FSM and EFSM are different modeling formalisms, they share the same structure of states, transitions, and alphabet. Whether they both comply with the requirements of the PUS service is also manually inspected.

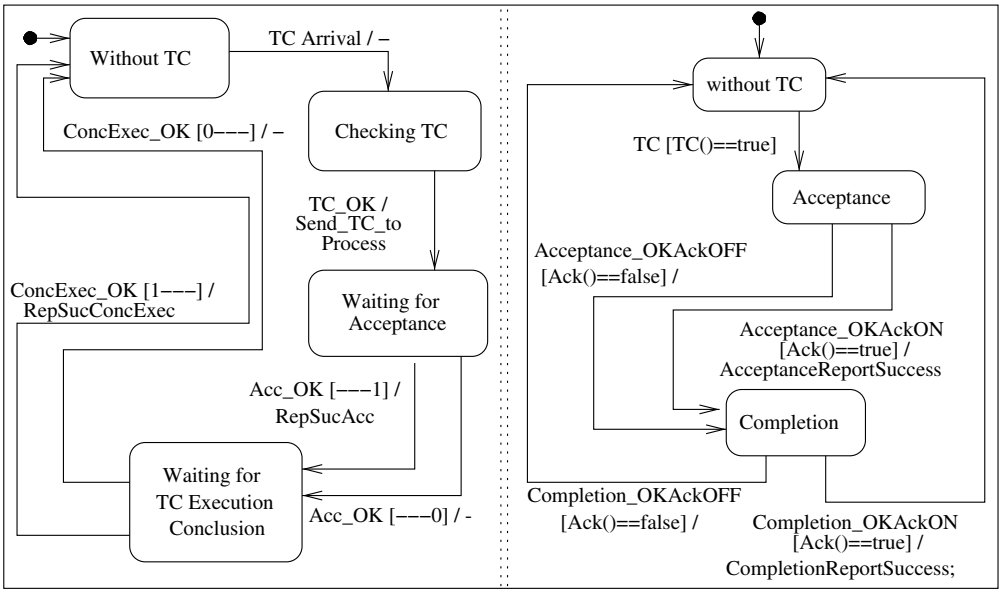
The manual models representing the Telecommand Verification Service of the PUS document [6] were elaborated by three different researchers/engineers, who considered different sets of mandatory and optional functionalities. They were obtained from [7–9] and are described ahead. The models presented in [9] were created to illustrate the application of the Conformance and Fault Injection (COFI) testing methodology in space applications. This methodology describes several steps for manual transformation of a textual description into a set of FSMs and for automatic generation, from the FSM of a set of conformance test cases to be executed against the corresponding service implementation. The service's behavior is represented by four different FSM classes: normal behavior, expected exception, sneak path, and fault tolerance. The sneak path is a correctly specified input occurring at an unexpected moment; fault tolerance paths deal with inputs modified by hardware faults, supposedly dealt with by fault tolerance mechanisms [9], but the information used to compose the models is selected according to the class.

In [7], the author investigated the advantages of model checking in the development of embedded space software. The investigation involved two different verification techniques for space applications: 1) a model-checking technique based on the UPPAAL tool [27] and 2) the COFI methodology [9]. To evaluate these techniques, a case study was conducted with two real software implementations of the Telecommand Verification Service.

Véras [8] presents a systematic procedure to evaluate the quality of software requirements for space applications that adopt the PUS standard. A benchmark composed of a checklist was provided and applied to real space projects (one academic and two industrial projects). The COFI methodology and the corresponding FSMs were used to create the checklist.

These three studies brought about FSMs that were manually modeled. In this paper, they are compared with the semi-automatically generated EFSMs. It is important to clarify that the complete natural language parameters from the semi-automatically generated models are suppressed to facilitate the comparison.

The models discussed in this section are named after their author. Models from Pontes [7], Véras [8], and Ambrosio [9] are named, respectively, Pi, Vi, and Ai, where i is an integer, and the semi-automatic generated models are named Gi.



a) Manually generated model **b) Semi-automatic generated model**

Fig. 2 Normal flow to Telecommand Verification Service. Adapted from [7,26].

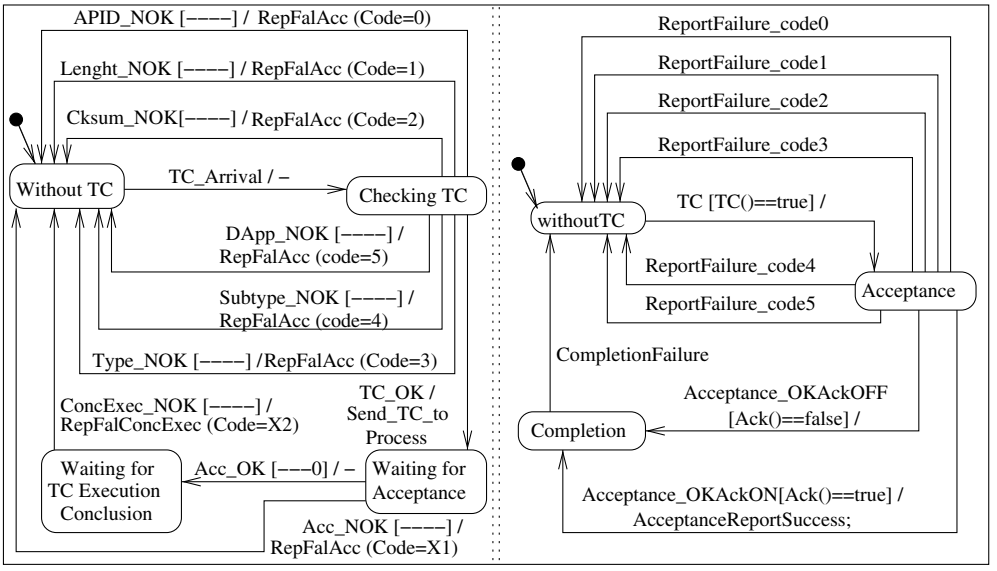
The first comparison is between P1 (Fig. 2a), a model from [7] representing the normal flow of the Telecommand Verification Service, and the corresponding semi-automatically generated model, hereafter referred to as G1 (Fig. 2b). Model G1 was already presented in [11], but here the natural language conditions are complete, as in [26]. In all compared models, TC refers to telecommand.

Comparing these models, it can be observed that there is an additional state in P1: Checking TC. This state, in G1, is represented as a transition parameter. The standard document describes the verification that must take place for a telecommand to be accepted, and in the method presented here, this information is automatically extracted as trigger conditions, which is allowed in EFSMs. The manual models are FSMs, in which parameters are not allowed, and this may be the reason for the additional state. In addition to this difference, the description of events, specifically sending reports about the success or failure of a stage of the service, is represented in different ways. Despite the differences, the main information described in the standard document is present in both models.

The second comparison is performed between models representing the exceptional flow of the Telecommand Verification Service: P2 (Fig. 3a) and G2 (Fig. 3b), the corresponding semi-automatically generated model. Besides the differences already mentioned in the comparison between P1 and G1, in P2, there are additional transitions, corresponding to the error codes X1 and X2, which are not present in G2. These transitions are allowed by the standard document, but they are mission specific, and since no specific mission was used in this research, they were not included in G2. A summary of the comparisons is presented in Table 1, where MM refers to manually generated models and SGM refers to semi-automatic generated models.

The third comparison is between the model from [8], representing the normal behavior of the Telecommand Verification Service, hereafter referred to as V1 (Fig. 4a) and the corresponding semi-automatically generated model, hereafter referred to as G3 (Fig. 4b). Model G3 was already presented in [10], but in this paper, once more, the natural language conditions are described, as in [26].

The differences observed comparing V1 and G3 are basically the same shown in the comparison between P1 and G1 and between P2 and G2. In addition to these differences, self-loops (transitions T7 and T8) can be observed in V1. Self-loops could be dealt with by the proposed method if the



a) Manually generated model **b) Semi-automatic generated model**

Fig. 3 Exceptional flow to Telecommand Verification Service. Adapted from [7,26].

Table 1 Summary of the comparisons

Characteristic	MM	SGM	Comments
	*P*1	*G*1	
Number of states	4	3	Additional state represented in G1 as trigger condition
Number of transitions	6	5	Consequence of the additional state
	*P*2	*G*2	
Number of states	4	3	Additional state
Number of transitions	11	10	Consequence of the additional state
	*V*1	*G*3	
Number of states	6	5	Additional state represented in G3 as trigger condition
Number of transitions	12	9	Consequence of the self-loops and additional state
	*V*2	*G*4	
Number of states	6	5	Additional state represented in G4 as trigger condition
Number of transitions	15	16	Additional information in transitions resulting from successful reports

information about self-loops were explicit in the text. However, the way this information is described in the text directly interferes with how to handle this situation.

The exceptional flow is also represented in [8], and the model is referred to hereafter as V2 (Fig. 5a). The corresponding semi-automatically generated model is referred to hereafter as G4 (Fig. 5b). Models V2 e G4 are very similar. The main differences are related to the successful reports available for them, at each stage of the processing; because success reports are optional, the manual models do not represent them. Nonetheless, the semi-automatically generated models always do since the user may use them to manually refine the model.

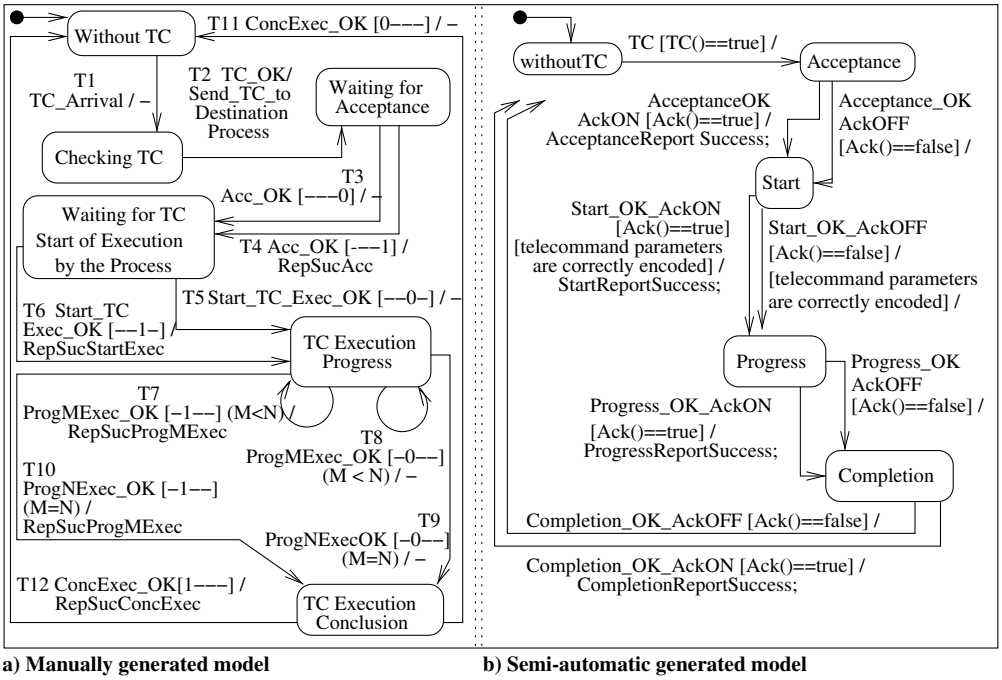
The models available in [9] were elaborated following the COFI methodology, which indicates the definition of test purposes for each selected service. The models can be quite different from the semi-automatically generated ones, but the information about the system's behavior is the same. The normal flow for the first purpose, represented in the model and hereafter referred to as A1 (Fig. 6a), is to test the correctness of the received telecommand and the viability of its execution, i.e., to verify whether the telecommand is correctly accepted. The normal flow for the second purpose, represented in the model referred to hereafter as A2 (Fig. 7a), is to verify the execution stages of the telecommand, i.e., after the acceptance, the start and the conclusion of the telecommand execution. The exceptional flow for the first purpose is represented in the model and is hereafter referred to as A3 (Fig. 8a) and for the second purpose is represented in the model and hereafter referred to as A4 (Fig. 9). All models available in [9] were originally described in Portuguese, but here they were translated to English.

Model G5 (Fig. 6b), semi-automatically generated, represents the normal flow of the Telecommand Verification Service for purpose 1, with the natural language parameters to ease understanding. Those parameters are text fragments extracted from the PUS document [6] and used in the model's composition.

The comparison between models A1 and G5 is very interesting. The set of verifications that in G5 are taken as guards associated to a single transition in A1 are sequentially performed through the different transitions and correspond to the FSM states. These models represent the configuration of the service only with mandatory features.

The second purpose defined in [9] is represented in model A2 (Fig. 7a). Model G6 (Fig. 7b), semi-automatically generated, represents the normal flow of the Telecommand Verification Service for purpose 2, and, as before, the natural language parameters will be presented. Those parameters are also text fragments extracted from the PUS document [6] and used in the models composition, as in model G5.

The second purpose, according to [9], is to monitor the stages of the telecommand execution, sending reports on demand. This objective could be interpreted as the complete representation of each stage of the service. However, the model presented in A2 shows only two stages: the



a) Manually generated model **b) Semi-automatic generated model**

Fig. 4 Normal flow to Telecommand Verification Service. Adapted from [8,26].

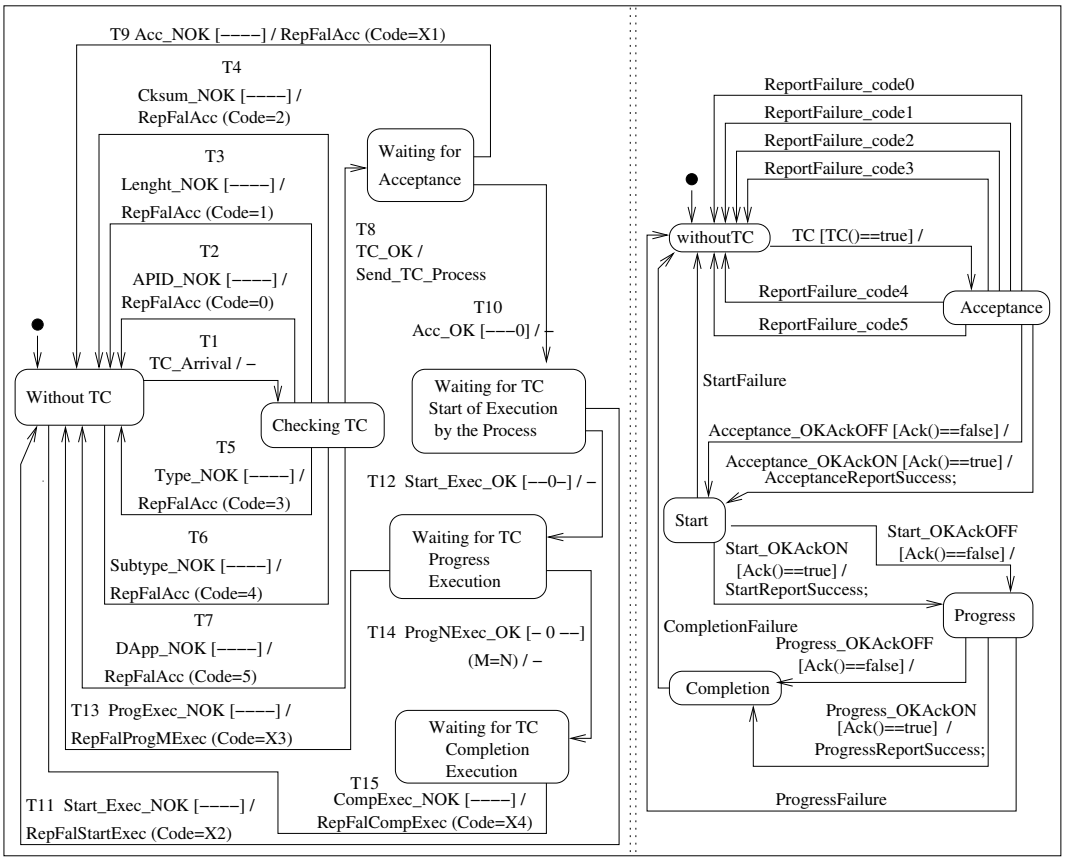


Fig. 5 Exceptional flow to Telecommand Verification Service. Adapted from [8,26].

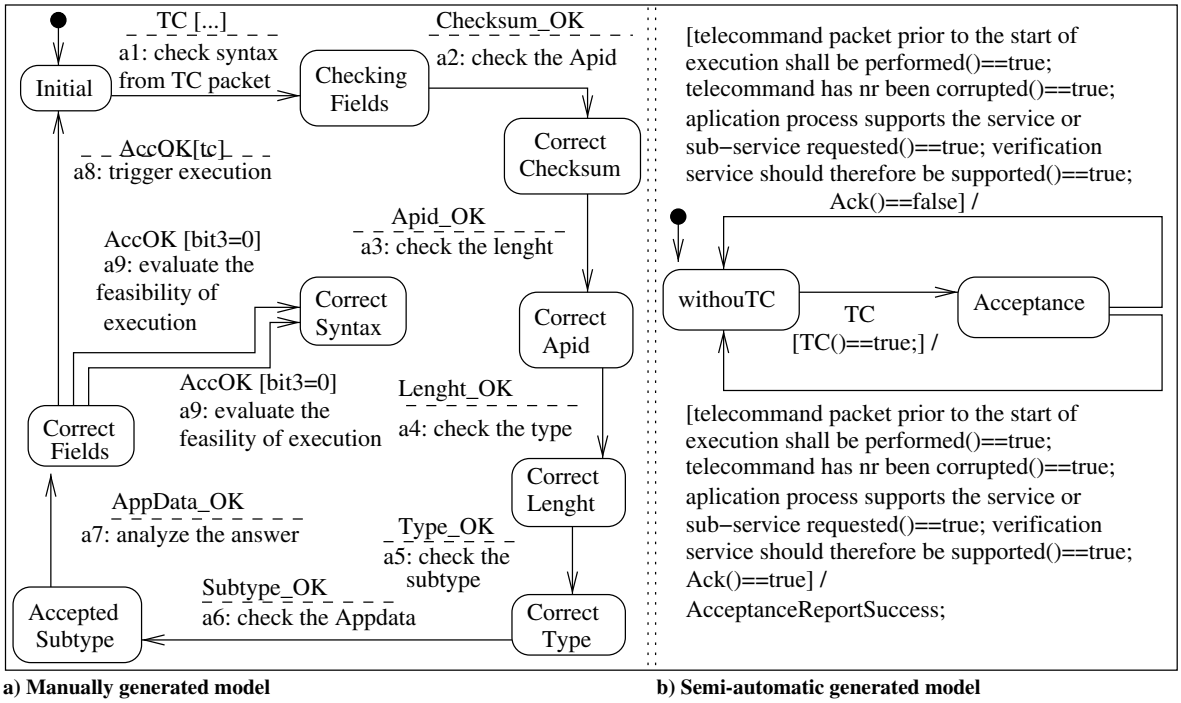
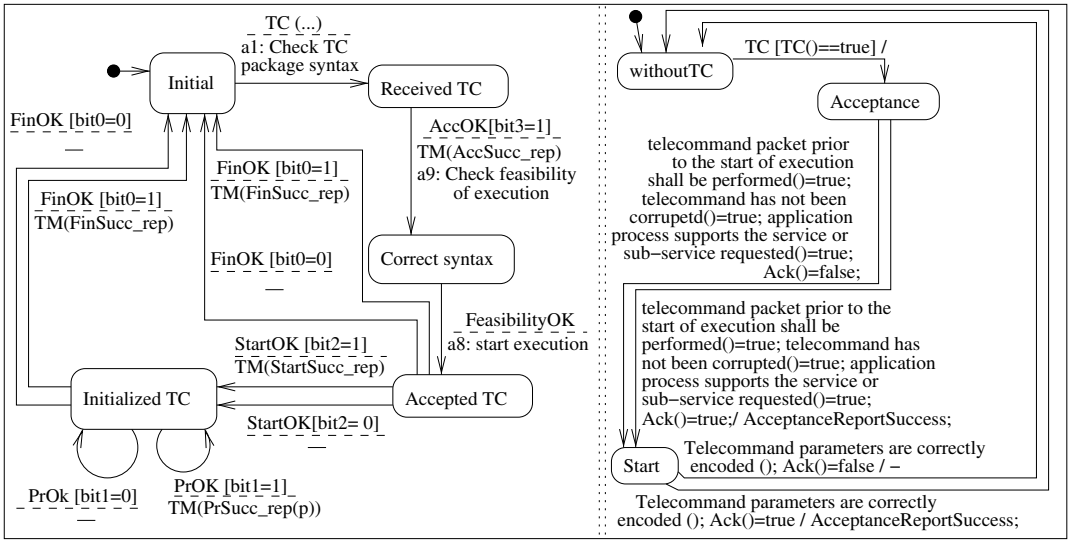


Fig. 6 Normal flow to Telecommand Verification Service: first purpose. Adapted from [9,26].

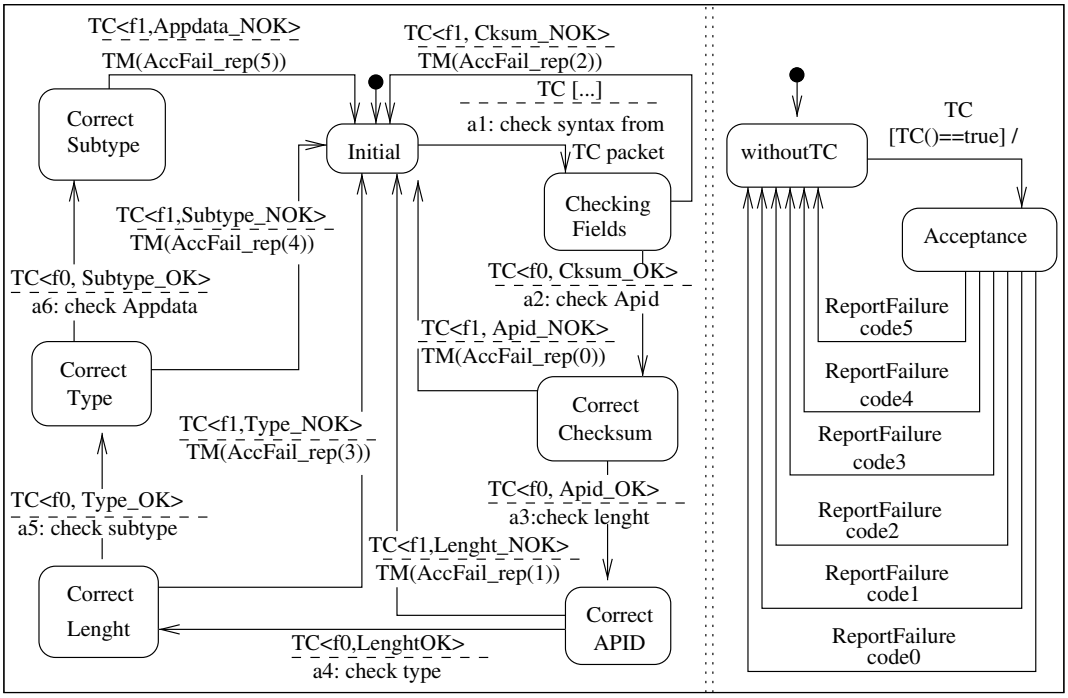
acceptance and the start of the execution. To compare both models, the user selects only one optional feature, the start of the execution, and a model, representing the mandatory and this optional feature, is semi-automatically generated (G6).

Comparing the models, we can observe two different states in A2: Received TC and CorrectSyntax. These states correspond to guards presented in G6. The states Accepted TC and Initialized TC in A2 correspond to the Acceptance and Start states in G6. The self-loops PrOK(p) bit1 $=$ 1 and PrOK(p) bit1 $=$ 0 are not represented in G6, but this is not a self-loop problem.



a) Manually generated model **b) Semi-automatic generated model**

Fig. 7 Normal flow to Telecommand Verification Service: second purpose. Adapted from [9,26].



a) Manually generated model **b) Semi-automatic generated model**

Fig. 8 Exceptional flow to Telecommand Verification Service: first purpose. Adapted from [9,26].

To satisfy the second purpose, the semi-automatically generated model must be complete, with all stages described in the standard document. This is the model represented in G3 (Fig. 4). The self-loop is represented by the state Progress, indicating the stage in which the progress of the execution is controlled. The conclusion of the execution is represented, in A2, with the transition FinOK, while in G3, it is represented as the last state Completion.

Model A3, representing the exceptional flow for the first purpose defined in [9], is presented in Fig. 8a. Model G7 (Fig. 8b), semi-automatically generated, represents the exceptional flow for purpose 1, and, as before, the natural language parameters will be presented to ease understanding. Those parameters are also text fragments extracted from the PUS document [6], as in models G5 and G6, and are used in the composition of the model.

In A3, there is a transition from any state to the Initial state. These transitions are represented in G7 as transitions from the Acceptance state to the WithoutTC state. They represent guards not satisfied, which cause expected errors, described in the standard document.

Figure 9 presents model A4, representing the exceptional flow for the second purpose defined in [9]. The semi-automatically generated model for purpose 2 is complete, and it is presented in G4 (Fig. 5b).

V. Discussion

The FSMs manually modeled and presented here come from different projects, in different contexts, but they were obtained from the same PUS standard. This could be the main reason for the differences among the manual models elaborated by Pontes [7], Véras [8], and Ambrosio [9].

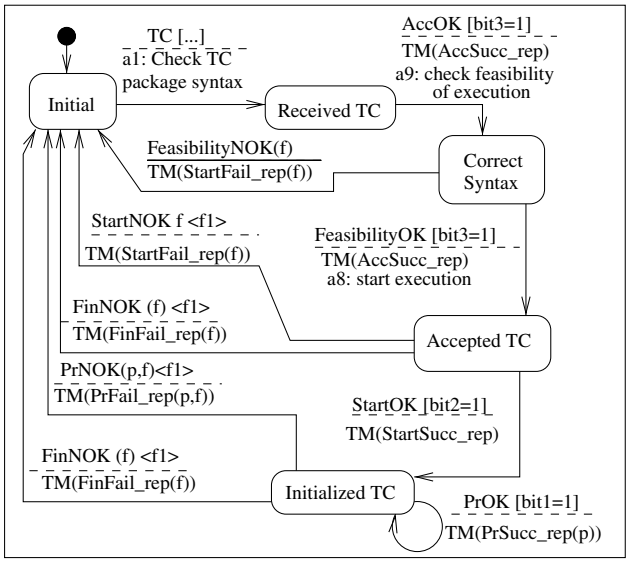


Fig. 9 Exceptional flow to Telecommand Verification Service: second purpose. Adapted from [9].

Nevertheless, they are very similar concerning the mandatory and optional features for the Telecommand Verification Service. The discussion is first about the differences between the manually created models and later about the differences between the semi-automatic generated models and the corresponding manual models.

Models P1 and V1 are very similar. The differences arise because P1 has only three states for the Telecommand Verification Service, the mandatory state and two optional states, whereas V1 may have four, the mandatory state plus three optional states. A1 and A3, however, are completely different for P1 or V1. In A1 and A3, a great deal of information described as trigger conditions in the standard document is modeled as states; moreover, the stages described for the service from the acceptance of the telecommand to completion of the execution are not even considered. A similar condition occurs in the comparison between P2 and V2 and between A2 and A4. Whereas P2 and V2 are very similar, A2 and A4 disregarded the stages described for the service and considered each cluster of data (verified before the acceptance of the telecommand) as a different state. The influence of the analyst's expertise in the final model is very clear in this example, and this may be harmful since information described in the standard document has not been considered. This evidence shows the importance of the proposed method and the usefulness of the developed tool to guide modeling in an automatic way. Concerning the manual and semi-automatic generated models, we first compare P1 (the normal flow) with G1 (the normal flow) and P2 (the exceptional flow) with G2 (the exceptional flow).

As previously stated, P1 considers only one additional capability described in the standard document, and the representation is in accordance with the description. However, we can observe a state named Checking TC before the state Acceptance. The verification of some parameters is described as a condition for the Acceptance stage to be completed. P1 is a FSM, and, as such, parameters are not allowed in transitions, as guard conditions do not exist in FSMs. The decision of representing the conditions through a state instead of trigger conditions may be influenced by the expertise of the author, and this kind of combination might influence the conformance (or nonconformance) of models that are manually generated.

Another possible source of problems and vagueness is the name of the transitions in P1: the segment [1 – – –] is not a guard since this model is not an EFSM. This segment is part of the identification of the transition, which could cause misinterpretations of the model. In G1, the text segment AckON or AckOFF represents exactly the same information, and the guards are natural language sentences describing the conditions that must be satisfied. The same can be observed in P2 and G2.

P2 represents the exceptional flow of the same situation modeled by P1, and it considers only one additional capability. It can be observed that the exceptional transitions are from Checking TC to Without TC, and in G2, the exceptional transitions are from Acceptance to Without TC. This is an effect of the additional state representing the guards to be checked. Since the conditions must be verified for the acceptance stage to be completed, in G2, the exceptional transitions are from Acceptance. In addition, there are extra exceptional transitions representing error codes X1 and X2. These transitions are not represented in G2; although they are described in the standard document, they must be used to represent errors according to the mission requirements.

The next comparison is between models V1 (the normal flow) and G3 (the normal flow) and between V2 (the exceptional flow) and G4 (the exceptional flow). It can be observed that V1 considers, besides the mandatory capability, all additional capabilities described in the standard document. The differences between V1 and G3 are very similar to those between P1 and V1: the addition of a state named Checking TC to verify the conditions for the acceptance stage to be completed and the inadequate name for the transitions.

In V1 there are two transitions, T7 and T8, that are self-loops. They are correct according to the standard document, but self-loops, as already mentioned in the literature, are difficult to deal with in (semi-)automatic model generation. In the proposed method, this kind of transition could be treated as if they were explicitly described in the text, but no statement can be made about the complexity of such processing.

In relation to V2 and G4, the differences are the extra transitions, the use of error codes starting with X, and the presence of self-loops. Some differences between P1, P2, V1, and V2 and G1, G2, G3, and G4 can be understood by noting that P1, P2, V1, and V2 are all FSMs, and the solution adopted by the analyst to represent guards was the addition of a state before the first stage described by the document. The use of inadequate transition names must be revised to avoid misinterpretation.

The last comparison is between models A1 (the normal flow for purpose 1) and G5 (representing the normal flow for purpose 1), between A2 (the normal flow for purpose 2) and G6 (representing the normal flow for purpose 2), between A3 (the exceptional flow for purpose 1) and G7 (representing the exceptional flow for purpose 1) and between A4 (the exceptional flow for purpose 2) and G4. G4 represents the exceptional flow for purpose 2. The influence of the analyst is clearly observed in A1: the analyst adds a state for each condition that must be satisfied for the completion of the acceptance stage. In addition, there is a conflict between states Correct Fields and Correct Syntax; since the execution of the telecommand will be triggered by Correct Fields, the objective of the Correct Syntax state is not clear. The names of the transitions are more adequate than those used in V1 and V2 [8] and P1 and P2 [7]; nonetheless, two transitions present the same problem of the cited models: transitions are named using a common notation to represent guards, and this can cause misinterpretation. A nonconformance with the requirements described

in the PUS document can be observed: the reports for the acceptance stage are defined as a minimum capability, and it must be always available. In A1, this stage is not considered or represented as a state; the information about the reports is represented as transitions between Correct Fields and Correct Syntax before the complete acceptance of the telecommand.

The influence of the analyst can be observed again in A2; although the service being represented is the same, the actual representation is completely different. The conditions are not represented; the execution is triggered by the Correct Syntax state and not by the Correct Fields state, as in A1. Two additional states are represented: the acceptance of the telecommand (Accepted TC) and the start of the execution Initialized TC. The stages of start, progress, and completion of the execution of the telecommand are described in the PUS document, but, in this model, only two stages are considered: the acceptance and the start of execution. The reports for each stage, even those that are not represented, are considered and correctly encoded. However, the transition's names show the same problem of inadequacy.

Although semi-automatic generated models may seem more complete, the obtained models need to be refined by human analysts, who can add mission-specific information, for example.

VI. Conclusions

The use of standard documents to guide the development of computational systems is highly recommended. However, as any document in natural language, standard documents may pose problems like inconsistencies and ambiguities and can lead to unsuccessful situations in the process of developing computational systems. The use of models is recommended, in an attempt to minimize these problems. Given that modeling is an error-prone task, this paper presents a method to semi-automatically generate EFSM from standard documents.

This paper compared the obtained results with manually generated models to illustrate that the semi-automatically generated models are in correspondence with manual ones and with the document. A working example was presented, as was a tool that was developed as a proof of concept. The standard document used in the example was the PUS document; specifically, the Telecommand Verification Service was selected. The semi-automatically generated models were compared with the manual models elaborated for the same service. Indeed, the semi-automatically generated models were shown to be in correspondence with the manual ones and with the PUS document for the same service.

Many differences among the compared models were identified, showing evidence for using semi-automatic generation, achieving a higher degree of conformance between the standard document and the model. However, although the semi-automatically generated models might avoid nonconformance information, they cannot treat self-loops or hierarchies yet. These two characteristics are not part of the Telecommand Verification Service, but they are found in other PUS services such as the onboard operations scheduling service. Preliminary work applying the presented method to those services showed that the treatment of these two characteristics depends on the quality of the document. In those cases, incompleteness and ambiguities issues hindered the successful application of the method.

Future work will focus on how to deal with these problems. The results are very encouraging, mainly because of the higher correspondence between the semi-automatically generated models and the standard document and the decrease of influence from the analyst's expertise in the semi-automatically generated models. The analyst's expertise is important in the preprocessing phases, such as in the elaboration of the features diagram; however, this knowledge should not influence the modeling of the first version of the EFSM before human refinement, for example, by adding information not described in the standard document but considered as common sense. This can bring more uniformity to models used by different organizations in different projects, for development and testing processes. Yet, the manual refinement of the generated models is always necessary.

The intent is to extend the work to deal with different standard documents, so that the necessary changes can be made to the TXT2SMM tool, and evaluate the generated models according to the end user's viewpoint. A new set of services described in the PUS document will be selected. Once all PUS services have been modeled as EFSMs, a set of test cases automatically generated may also become part of a standard for conformance evaluation.

Acknowledgments

The authors would like to thank the University of Lavras, the University of Campinas, the National Institute for Space Research, the Aeronautics Institute of Technology, and all research agencies which, indirectly, supported this study.

References

[1] Ambriola, V., and Gervasi, V., "Processing Natural Language Requirements," *Proceedings 12th IEEE International Conference Automated Software Engineering*, IEEE Publ., Piscataway, NJ, 1997, pp. 36–45.

[2] Lutz, R. R., and Ampo, Y., "Experience Report: Using Formal Methods for Requirements Analysis of Critical Spacecraft Software," *Proceedings for the 19th Annual Software Engineering Workshop*, NASA Goddard Space Flight Center, Greenbelt, MD, 1994, pp. 231–248.

[3] Yue, T., Briand, L., and Labiche, Y., "An Automated Approach to Transform Use Cases into Activity Diagrams," *Modelling Foundations and Applications, LNCS*, Vol. 6138, edited by T. Kühne, B. Selic, M. Gervais, and F. Terrier, Springer–Verlag, Berlin, 2010, pp. 337–353. doi:10.1007/978-3-642-13595-8_26

[4] Deeptimahanti, D. K., and Babar, M. A., "An Automated Tool for Generating UML Models from Natural Language Requirements," *Proceedings of the 2009 IEEE/ACM International Conference on Automated Software Engineering (ASE '09)*, IEEE Computer Soc., Washington, D.C., 2009, pp. 680–682. doi:10.1109/ASE.2009.48

[5] Kof, L., "Translation of Textual Specifications to Automata by Means of Discourse Context Modeling," *Requirements Engineering: Foundation for Software Quality*, Lecture Notes in Computer Science, Vol. 5512, edited by M. Glinz, and P. Heymans, Springer–Verlag, Berlin, 2009, pp. 197–211. doi:10.1007/978-3-642-02050-6_17

[6] ECSS Working Group for Ground Systems and Operations, "Space Engineering: Ground Systems and Operations. Telemetry and Telecommand Packet Utilization," Rept. ECSS-E-70-41A, ESA Publications Division, 2003.

[7] Pontes, R., "Contribuições do Model Checking e da Metodologia CoFI para o Software Embarcado Espacial," Master's Thesis, Aeronautics Inst. of Technology, São José dos Campos, Brazil, 2011.

[8] Véras, P., "Benchmarking Software Requirements for Space Applications Documentation," Ph.D. Thesis, Aeronautics Inst. of Technology, São José dos Campos, Brazil, 2011.

[9] Ambrosio, A. M., "CoFI: Uma Abordagem Combinando Teste de Conformidade e Injeção de Falhas para Validação de Software em Aplicações Espaciais," Ph.D. Thesis, National Space Research Inst., São José dos Campos, Brazil, 2005.

[10] Greghi, J. G., Martins, E., and Carvalho, A. M. B. R., "Semi-Automatic Generation of Extended Finite State Machines from Natural Language Standard Documents," *2015 IEEE International Conference on Dependable Systems and Networks Workshops*, IEEE Publ., Piscataway, NJ, 2015, pp. 45–50. doi:10.1109/DSN-W.2015.17

[11] Greghi, J. G., Martins, E., and Carvalho, A. M. B. R., "Requirement's Variability in Model Generation from a Standard Document in Natural Language," *2015 International Conference on Software Engineering Advances*, IARIA, 2015, https://www.thinkmind.org/download.php?articleid=icsea_2015_6_10_10205.

[12] Pontes, R. P., Véras, P. C., Ambrosio, A. M., and Villani, E., "Contributions of Model Checking and CoFI Methodology to the Development of Space Embedded Software," *Empirical Software Engineering*, Vol. 19, No. 1, 2014, pp. 39–68.
doi:10.1007/s10664-012-9215-y
[13] Berry, D. M., and Kamsties, E., *Ambiguity in Requirements Specification*, Springer–Verlag, New York, 2004, pp. 7–44.
doi:10.1007/978-1-4615-0465-8_2
[14] Deeptimahanti, D. K., and Sanyal, R., "Semi-Automatic Generation of UML Models from Natural Language Requirements," *Proceedings of the 4th India Software Engineering Conference*, Assoc. of Computing Machinery, New York, 2011, pp. 165–174.
doi:10.1145/1953355.1953378
[15] Kof, L., and Penzenstadler, B., "Faster from Requirements Documents to System Models: Interactive Semi-Automatic Translation," *Requirements Engineering Efficiency Workshop, 17th International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ 2011)*, ICB-Research Rept. n. 44, Essen, Germany, 2011, pp. 14–25, http://www.icb.uni-due.de/fileadmin/ICB/research/research_reports/ICB-Report-No44.pdf.
[16] Ali, S., Yue, T., Briand, L., and Walawege, S., "A Product Line Modeling and Configuration Methodology to Support Model-Based Testing: An Industrial Case Study," *Model Driven Engineering Languages and Systems*, Lecture Notes in Computer Science, Vol. 7590, edited by R. France, J. Kazmeier, R. Breu, and C. Atkinson, Springer–Verlag, Berlin, 2012, pp. 726–742.
doi:10.1007/978-3-642-33666-9_46
[17] Yue, T., and Ali, S., "Bridging the Gap Between Requirements and Aspect State Machines to Support Non-Functional Testing: Industrial Case Studies," *Modelling Foundations and Applications*, Lecture Notes in Computer Science, Vol. 7349, edited by A. Vallecillo, J. Tolvanen, E. Kindler, H. Störrle, and D. Kolovos, Springer–Verlag, Berlin, 2012, pp. 133–145.
doi:10.1007/978-3-642-31491-9_12
[18] Yue, T., Briand, L., and Labiche, Y., "A Use Case Modeling Approach to Facilitate the Transition Towards Analysis Models: Concepts and Empirical Evaluation," *Model Driven Engineering Languages and Systems*, Lecture Notes in Computer Science, Vol. 5795, edited by A. Schürr, and B. Selic, Springer–Verlag, Berlin, 2009, pp. 484–498.
doi:10.1007/978-3-642-04425-0_37
[19] Yue, T., Briand, L. C., and Labiche, Y., "aToucan: An Automated Framework to Derive UML Analysis Models from Use Case Models," *ACM Transactions on Software Engineering and Methodology*, Vol. 24, No. 3, 2015, pp. 1–52.
doi:10.1145/2776776
[20] Vieira, N. J., *Introdução aos Fundamentos da Computação: Linguagens e Máquinas*, Pioneira Thomson Learning, Brazil, 2005, pp. 67–112.
[21] Bochmann, G., and Petrenko, A., "Protocol Testing: Review of Methods and Relevance for Software Testing," *Proceedings of the 1994 ACM SIGSOFT International Symposium on Software Testing and Analysis*, Assoc. for Computing Machinery, New York, 1994, pp. 109–124.
[22] Kang, K., Cohen, S., Hess, J., Novak, W., and Peterson, A., "Feature-Oriented Domain Analysis (FODA) Feasibility Study," Tech. Rept. CMU/SEI-90-TR-021, Software Engineering Inst., Carnegie Mellon Univ., Pittsburgh, PA, 1990.
[23] Manning, C. D., and Schütze, H., *Foundations of Statistical Natural Language Processing*, MIT Press, Cambridge, MA, 1999.
[24] Toutanova, K., Klein, D., Manning, C. D., and Singer, Y., "Feature-Rich Part-of-Speech Tagging with a Cyclic Dependency Network," *Proceedings of the NAACL '03*, Vol. 1, Assoc. for Computational Linguistics, Stroudsburg, PA, 2003, pp. 173–180.
doi:10.3115/1073445.1073478
[25] Rapp, C. W., "The State Machine Compiler," http://smc.sourceforge.net/ [retrieved 2 Dec. 2016].
[26] Greghi, J., "Geração Semiautomática de Máquinas Finitas de Estados Estendidas a Partir de Documento de Padronização em Língua Natural," Ph.D. Thesis, Inst. of Computing, Univ. of Campinas, Campinas, Brazil, 2015, http://repositorio.unicamp.br/jspui/handle/REPOSIP/275585.
[27] Behrmann, G., David, A., and Larsen, K. G., *A Tutorial on Uppaal*, Springer–Verlag, Berlin, 2004, pp. 200–236.
doi:10.1007/978-3-540-30080-9_7

M. D. Davies
Associate Editor